DATA ENGINEERING

Kafka Deep Dive Visual Guide

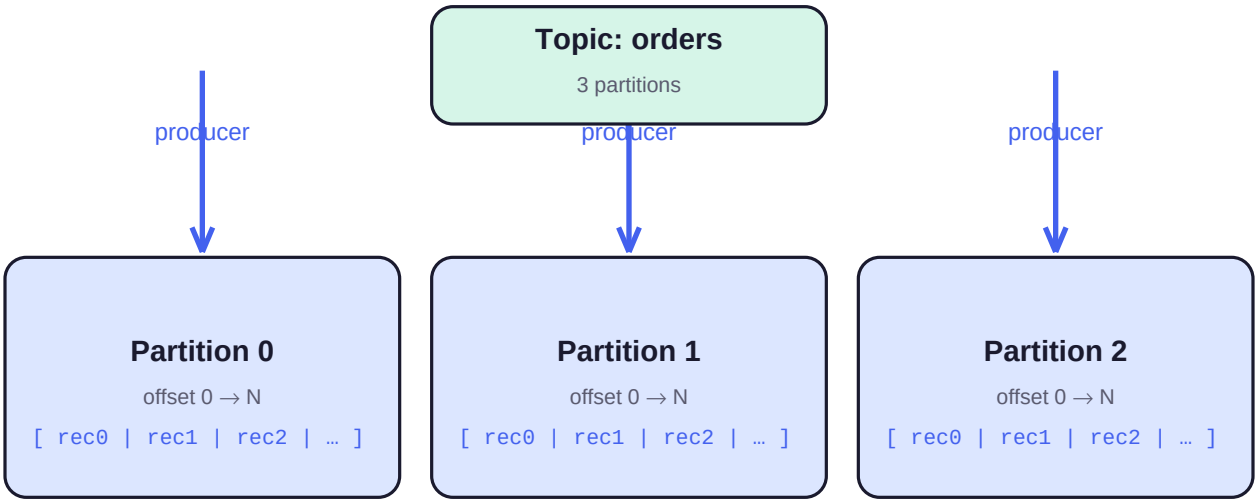
Partitions, offsets, groups, and exactly-once.
How the distributed log actually works.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Topic Anatomy: Partitions & Offsets

A topic is a partitioned, append-only log. Order is guaranteed only within a partition.

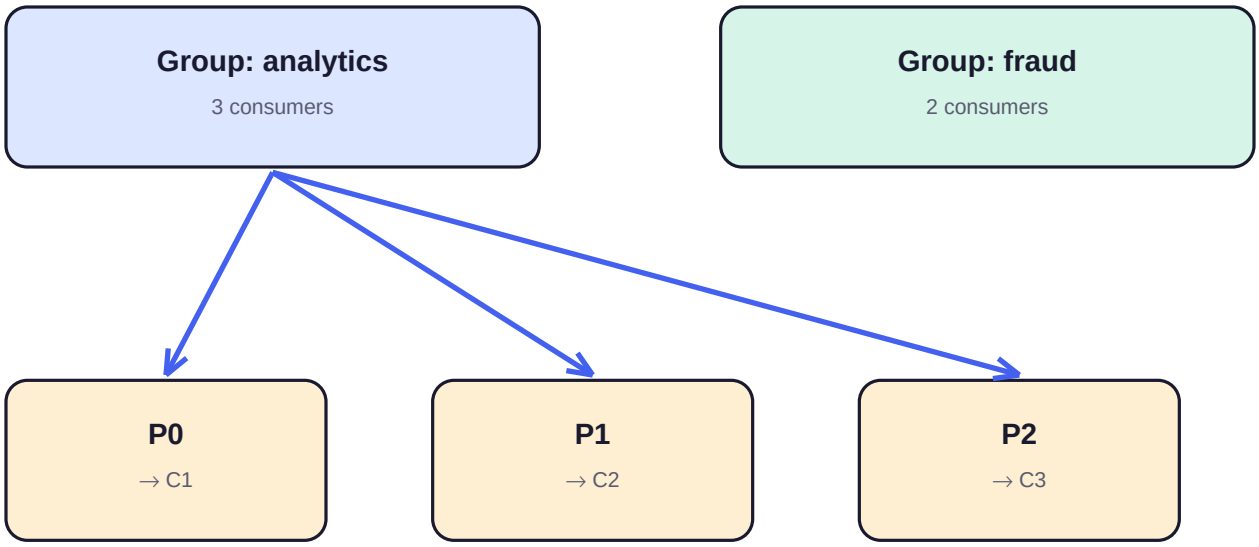


Each record lands in one partition (by key hash or round-robin) and gets a sequential offset — its position in that partition.

STEP 2/6

Consumer Groups & Rebalancing

Within a group, each partition is read by exactly one consumer — parallelism equals partition count.



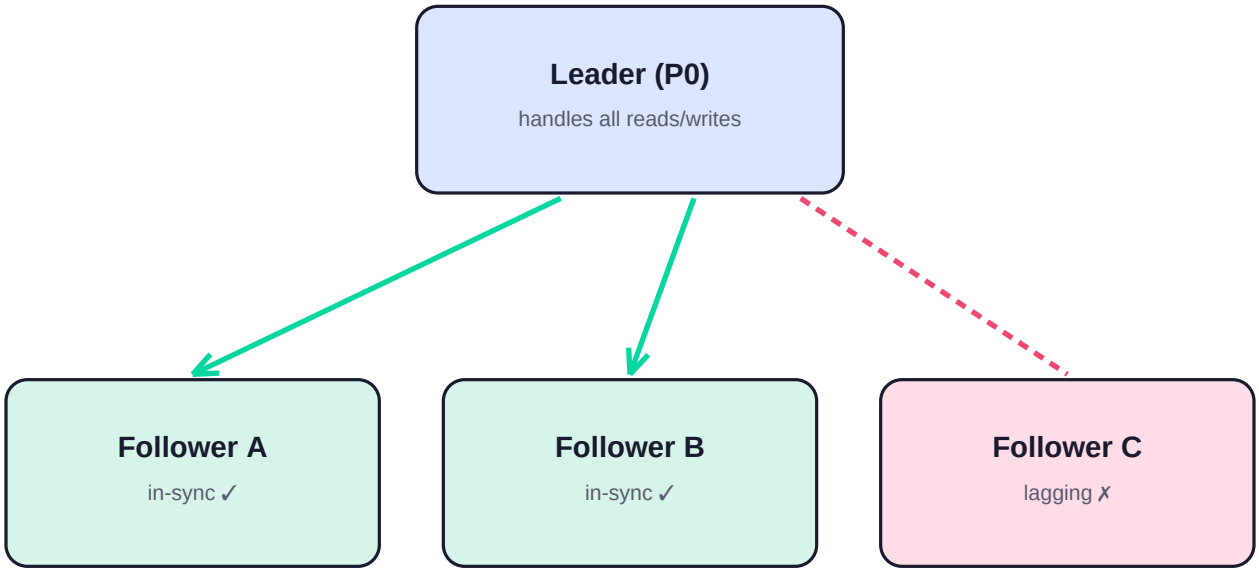
Two groups each get EVERY record independently. Add a consumer and partitions rebalance — a brief pause while ownership is reassigned.

- Max useful consumers in a group = partition count
- A consumer crash triggers rebalance; its partitions move to survivors
- Sticky assignment minimizes partition movement between rebalances

STEP 3/6

Replication: ISR Keeps Data Safe

Every partition has a leader and followers. The In-Sync Replica set is what protects you.



A write is 'committed' when all ISR members have it. If the leader dies, an in-sync follower is elected — no committed data lost.

- min.insync.replicas: minimum ISR size before a write is accepted
- acks=all: producer waits for full ISR acknowledgment
- Out-of-sync followers can rejoin by catching up from the leader

STEP 4/6

Delivery Semantics: The Trio

At-most-once, at-least-once, exactly-once — pick your guarantee.

At-most-once

Commit offset BEFORE processing. Crash = lost record.

At-least-once

Commit offset AFTER processing. Crash = duplicate record.

Exactly-once

Idempotent producer + transactions. One effect, even on retry.

Exactly-once in Kafka = idempotent producer (dedupe by sequence) + transactional writes (atomic commit of records + offsets).

STEP 5/6

Retention & Log Compaction

Kafka is storage, not just a pipe. Old segments age out — or compact to the latest key.

Time retention

delete segments older than N days

Log compaction

keep only the latest value per key

```
k1=v1   k1=v2   k2=a   k1=v3
```

→ **k2=a k1=v3**

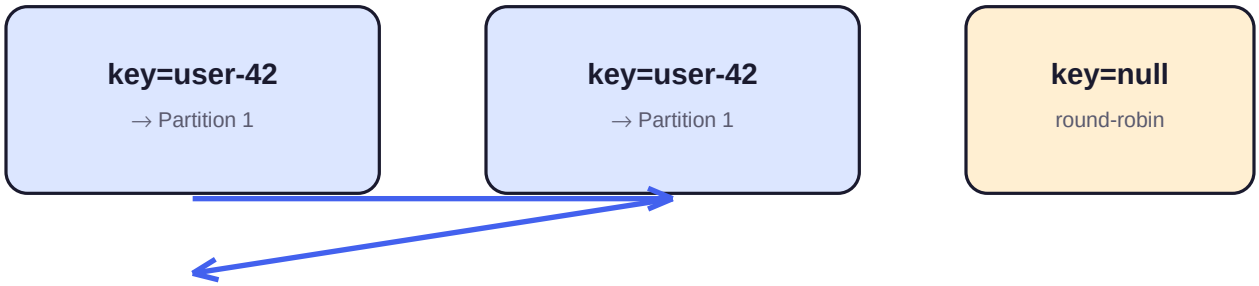
Changelog topics: rebuild current state from the log.

Compaction turns a topic into a durable key-value snapshot — the foundation of Kafka Streams state stores and CDC sinks.

STEP 6/6

Keys: Order Where It Matters

Same key → same partition → ordered processing per entity.



Keyed records give per-entity ordering (all events for user-42 processed in order). Null keys spread load evenly but sacrifice ordering.

- Choose the key from the entity you must serialize on
- Changing partition count re-hashes keys — plan partition counts upfront
- Hot keys create hot partitions: split the key or accept the skew

STEP ✓I✓

Key Takeaways

Kafka Deep Dive — Remember

- Topics are partitioned append-only logs; ordering holds within a partition, not across them.
- A consumer group splits partitions across consumers — add partitions to scale, not just consumers.
- The ISR (in-sync replicas) is the durability contract: committed = replicated to all ISR members.
- Exactly-once = idempotent producer plus transactions; at-least-once is the practical default.
- Log compaction keeps the latest value per key, turning topics into replayable state snapshots.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev